A Tour of C++

Mihir Rao

Contents

1 The Basics **2**
- 1.1 Programs and the compilation model . . . 2
- 1.2 Functions and overloading . . . 3
- 1.3 Types, values, and arithmetic . . . 3
- 1.4 Initialization and type deduction . . . 4
- 1.5 Scope and lifetime . . . 5
- 1.6 Constants and compile-time computation . . . 5
- 1.7 Pointers, arrays, and references . . . 6
- 1.8 Tests and control flow . . . 7
- 1.9 Mapping to hardware: copying values and sharing objects . . . 8

Chapter 1: The Basics

1.1 Programs and the compilation model

C++ combines a small core language with a standard library written largely in C++ itself. Types, expressions, functions, and control flow belong to the core; strings, containers, algorithms, and streams come from the library. Using those library abstractions is part of ordinary C++ programming.

C++ is statically typed: the compiler must know the type of an entity at its point of use. A type determines which operations make sense and how an object is represented. Compiling source files produces object files; linking combines those files and the required libraries into an executable.

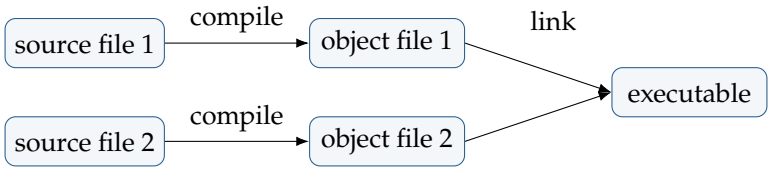


Figure 1: Compilation checks each source file; linking assembles the program. The executable targets a particular system. Source portability means that the same source can be compiled for different systems.

Execution of an ordinary program begins in the global function `main`. Its return type is `int`; returning zero signals successful completion. Reaching the end of `main` also returns zero. Braces group a function body, and `//` starts a comment extending to the end of the line.

```
#include <iostream>

int main()
{
    std::cout << "Hello, World!\n";
}
```

`std::cout` is the standard output stream; `std::` qualifies a name from the standard-library namespace. The stream's « operator writes its right operand. Double quotes delimit a string literal; \n represents a newline. Declarations must be made available before use, here by including `<iostream>`. The alternative notation `import std;` expresses access through a standard-library module; these notes use the header form so the examples do not depend on module setup.

1.2 Functions and overloading

A FUNCTION DECLARATION specifies its name, return type, and parameter types. A DEFINITION also supplies the body. Parameter names in a standalone declaration document intent but need not match the definition.

```
double square(double);          // declaration

double square(double x)         // definition
{
    return x*x;
}
```

The type here is `double(double)`. A return type of `void` means there is no result value. Calling a function initializes its parameters from the arguments: types are checked, and permitted conversions occur. Thus `square(2)` converts the integer to `double`, whereas `square("two")` is ill-typed. Functions give names to meaningful operations and make their dependencies explicit. Keep each function focused on one logical task.

OVERLOADING gives the same name to functions with different parameter types. The compiler selects the best match using the call's arguments. `print(int)` and `print(double)` can express the same operation for different types. Overloads should share a conceptual meaning. When neither candidate is better, the call is an error:

```
void print(int, double);
void print(double, int);
// print(0, 0);  // ambiguous: each overload needs one conversion
```

1.3 Types, values, and arithmetic

A TYPE specifies possible values and operations. An OBJECT is memory holding a value of some type; a VALUE is a bit pattern interpreted according to that type. A VARIABLE is a named object. A declaration introduces an entity and its type, as in `int inch;`.

Type	Role
`bool`	Truth values `true` and `false`
`char`	A character, such as `'x'`
`int`	Signed integer arithmetic
`double`	Double-precision floating-point arithmetic
`unsigned`	Unsigned integer representation, useful for bit operations

Sizes and ranges depend on the implementation. `sizeof(T)` reports the size in units of `char`; `sizeof(char)` is always one, while `sizeof(int)` is often four. Do not infer a portable size from a drawing of memory. Fixed-width aliases such as `int32_t` express an exact width when the implementation provides them.

Integer literals are decimal unless prefixed: `0b` denotes binary, `0x` hexadecimal, and an initial `0` octal. A decimal point or an exponent marks a floating-point literal, as in `3.14` or `314e-2`. Single quotes can separate digits for readability, as in `1'000'000`.

Arithmetic uses +, -, *, /, and integer remainder %. Comparisons ==, !=, <, <=, >, and >= produce truth values. Distinguish assignment = from equality testing ==. The logical operators &&, ||, and ! combine or negate conditions. Bitwise &, |, ^, and ~ operate on the individual bits of integral operands.

Mixed arithmetic can introduce conversions: adding an `int` to a `double` performs floating-point addition. Assigning the result back to an `int` can lose the fractional part. Compound assignments such as `x += y` update a variable; `++x` and `--x` increment and decrement it.

Evaluation order is a separate issue from the value of an expression. For built-in && and ||, the left operand is tested first, and the right operand is skipped when it cannot change the result. Assignment evaluates its right operand before its left. Do not assume left-to-right evaluation of `f(x) + g(y)` or of the arguments in `h(f(x), g(y))`. Keep expressions simple when evaluation has side effects.

1.4 Initialization and type deduction

Initialization supplies an object's initial value. Prefer a braced initializer when explicitly naming the type: it rejects narrowing conversions that might silently lose information under other initialization syntax.

```
double d1 = 2.3;
double d2 {2.3};
int i1 = 7.8;          // accepted, but the value becomes 7
// int i2 {7.8};       // error: narrowing conversion

auto b = true;        // bool
auto ch = 'x';        // char
auto i = 123;         // int
auto d = 1.2;         // double
```

`auto` asks the compiler to deduce a static type from the initializer; it does not postpone type checking to run time. It avoids repeating long or obvious type names. State the type explicitly when a particular range or precision matters, when the initializer's type is unclear, or when a widely

visible declaration benefits from spelling it out. Introduce variables when a suitable initial value is available. An uninitialized fundamental local variable has no useful value to read.

1.5 Scope and lifetime

Scope concerns where a name can be used. Lifetime concerns when its object exists. A local name belongs to a function or block; parameter names are local too. A member name belongs to a class. A namespace member belongs to its namespace; the outermost namespace is the global namespace. Keep names in the smallest useful scope.

An ordinary automatic local object is initialized when its declaration is executed and destroyed when its block is exited. A member's lifetime is tied to its containing object. Namespace objects generally last until program termination. Dynamically allocated objects created by `new` remain until destroyed by `delete`; the disappearance of a local pointer name does not by itself destroy the pointed-to object. Objects need not have names: temporaries and dynamically allocated objects are examples.

1.6 Constants and compile-time computation

`const` promises that a value cannot be changed through that name or access path. Its initializer may run at run time. A `constexpr` variable requires a constant-expression initializer that can be evaluated at compile time. These are different promises.

```
constexpr int dmv = 17;
int var = 17;
constexpr double square(double x) { return x*x; }

constexpr double max1 = 1.4*square(dmv);
const double max3 = 1.4*square(var);  // runtime input is allowed
// constexpr double max2 = 1.4*square(var);  // not constant
```

A `constexpr` function can serve both compile-time and run-time calls. The arguments and the surrounding context determine whether a particular call is a constant expression. A `consteval` function instead requires compile-time evaluation for ordinary calls:

```
consteval double square2(double x) { return x*x; }
constexpr double bound = square2(17);
// const double other = square2(var);  // runtime argument forbidden
```

Compile-time computation can use loops and local variables. For example, `nth(x,n)` can multiply an accumulator by `x` repeatedly for `n` iterations, with the precondition `n >= 0`. Constant evaluation restricts the operations that may be performed; merely declaring a function `constexpr` does not make all its run-time calls pure. Constant expressions are required in places such as built-in array

bounds, `case` labels, and `constexpr` initializers. Named constants also make intent clearer than unexplained literal values.

1.7 Pointers, arrays, and references

An ARRAY stores a contiguous sequence of elements of one type. `char v[6]` has indices zero through five. Its bound is a constant expression. A POINTER holds an address: `char* p = &v[3]` points to the fourth element. In an expression, unary `&` takes an address and unary `*` accesses the pointed-to object.

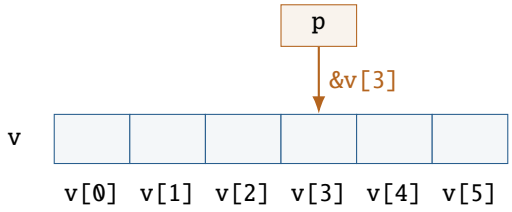


Figure 2: A pointer identifies an element; it does not contain the array. Here `*p` accesses `v[3]`, and incrementing `p` moves it to `v[4]`. This is a schematic, not a choice of numeric addresses.

A REFERENCE, declared with `T&`, is an alias for an existing object. It must be initialized and cannot later be rebound. Access through a reference needs no explicit dereference operator.

```
int v[] {0, 1, 2, 3};
for (auto x : v) {       // x is a copy of each element
    // reading x reads the copy
}
for (auto& x : v)        // x refers to each actual element
    ++x;
```

A range-`for` expresses traversal directly. With `auto x`, changing `x` changes only the copy. With `auto& x`, changing it changes the element. The same distinction matters for function parameters: `void sort(vector<double>& v)` can rearrange the caller's vector, while `double sum(const vector<double>& v)` can read it without copying and without modifying it through `v`.

Declaration	Meaning
`T a[n]`	Array of `n` objects of type `T`
`T* p`	Pointer to `T`
`T& r`	Reference to `T`
`T f(A)`	Function taking `A` and returning `T`

`nullptr` represents the absence of a pointed-to object. Use it instead of `0` or `NULL` for this purpose, and check for it before dereferencing a possibly null pointer. A reference has no corresponding valid null state.

```
int count_x(const char* p, char x)
{
    if (p == nullptr)
        return 0;
    int count = 0;
    while (*p) {
        if (*p == x)
            ++count;
        ++p;
    }
    return count;
}
```

The precondition is that `p` is null or points into a valid zero-terminated character array. The loop stops at the zero character, not at the end of storage inferred from the pointer. `const char*` permits reading characters but not changing them through `p`; the pointer itself can advance. Testing a number as a condition means testing it against zero; testing a pointer means testing it against `nullptr`.

1.8 Tests and control flow

`if` selects a branch; `while` repeats while a condition is true; `for` packages initialization, a continuation test, and an update. `std::cin » answer` reads into `answer`, with its type determining the input interpretation. A `switch` selects among distinct constant `case` labels; `default` handles unmatched values. Without a matching label or a default, no branch runs.

`break` exits the switch, while `return` exits the whole function. Adjacent case labels can share a body. Without a terminating statement, execution continues into the next case, so make such control flow deliberate.

```
if (auto n = v.size(); n != 0) {
    // n is available here
} else {
    // n is available here too
}
// n is out of scope
```

An initializer inside an `if` limits a temporary name to that statement, including both branches. When the test is simply against zero, the shorter `if (auto n = v.size())` expresses the same condition.

1.9 Mapping to hardware: copying values and sharing objects

Fundamental arithmetic and memory access map closely to machine operations. The useful memory model is a sequence of locations holding typed objects; pointers identify locations, and arrays place elements contiguously. The compiler can optimize this representation while preserving program behavior.

For built-in values, assignment copies a value into an existing object. After `x = y`, the integers hold equal values but remain separate objects. After `p = q`, the pointer objects also remain separate, but hold the same address and therefore access the same target. Assignment through a reference copies into its target; it does not change its binding.

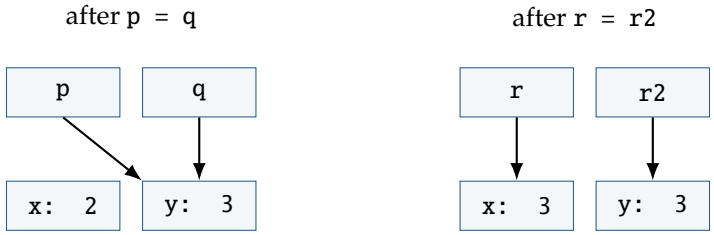


Figure 3: Both panels start with `x = 2` and `y = 3`. Initially `p` points to `x`, `q` to `y`, `r` refers to `x`, and `r2` to `y`. Pointer assignment changes an address; reference assignment changes a value. Arrows for references show bindings, not required storage layouts.

Initialization establishes a valid object; assignment changes one that already exists. Thus `int& r = x` binds `r` to `x`, whereas a later `r = y` assigns `y`'s value to `x`. An uninitialized reference is ill-formed. The distinction also matters for library types that own resources: assigning to an existing string or vector must account for its old state. Argument passing and return-value construction use initialization semantics.

Source. Bjarne Stroustrup, A Tour of C++, third edition, Chapter 1, pp. 1–20 (PDF pp. 14–33).